GDB debugging workshop

Session 3

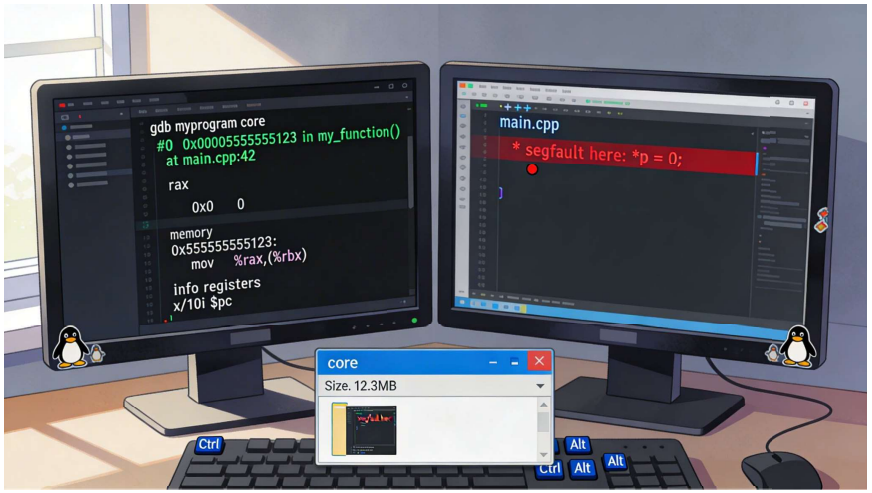
Session 1 + 2 :: recap

- Basic GDB CLI usage and commands
- Program data display
- Breakpoints
- GDB TUI
- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints

Session 3 agenda

- Core dump analysis
- Attaching to a running program
- Remote debugging
- Altering execution
- Checkpoints and Tracepoints

Coredump analysis using GDB



What is coredump?

- Snapshot of program memory at crash moment
- Contains address space, registers, call stacks
- Generated on fatal signals (e.g. segfault, abort)
- Used for post-mortem debugging without reproduction

What are the reasons for a crash? (1)

Hardware-Detected Violations

- **SIGSEGV (11) -** Segfault from invalid memory access (null pointers, buffer overflows, out-of-bounds).
- **SIGBUS (7) -** Bus error from misaligned memory access or invalid physical address.
- **SIGFPE (8) -** Floating-point exception (divide-by-zero, overflow, underflow, invalid operations).
- **SIGILL (4)** - Illegal instruction execution (corrupted code, wrong architecture).

What are the reasons for a crash? (2)

External or Internal (e.g. unhandled exception) Termination

- **SIGABRT (6) -** Abort signal from process calling abort() or failed assertion.
- **SIGTRAP (5) -** Trace trap (usually used by debugger, then not a crash. Rarely used otherwise).

What are the reasons for a crash? (3)

Resource Limit Violations

- **SIGXCPU (24) -** CPU time limit exceeded (soft limit warning, then hard limit SIGKILL).
- **SIGXFSZ (25) -** File size limit exceeded when writing beyond ulimit.
- **SIGSTKFLT (16) -** Stack fault from stack overflow or underflow.

Kernel-Initiated Termination

- **Watchdog Timeout -** Systemd service timeout or container grace period expiration enforces SIGABRT.

Crashes that won't create a coredump

Forced Termination Signals

- **SIGKILL (9)** - Immediate kill signal that cannot be caught, blocked, or ignored.

Kernel-Initiated Termination

- **OOM Killer -** Out-of-memory, sends SIGKILL to process with highest oom_score.
- **Exceeding cgroup limits -** Memory or CPU cgroup resource limits exceeded, triggers SIGKILL.

Point of failure != Point of bug

Analyzing the coredump is not always trivial

The program crashes on a point where the OS detected a non-recoverable fault

This is not necessary the point where the bug hides (the root cause)!

GDB commands for debugging coredumps

- **`bt/backtrace`** - Show the call stack
- **`bt full`** - Show the call stack with arguments
- **`frame <frame#>`** - Go to frame # in the call stack
- **`up/down`** - Navigate the stack frames level up/down
- **`list`** - Shows the source code lines around the current execution point
- **`print <varName>`** - Show varName value

Core dump analysis example

`day02/01_core_dump_with_symbols`

Let’s do it together!

Coredump settings - core file size

`ulimit -c <SIZE> (or “unlimited”)`

Or, for permanent setting for all future shell terminals:

Edit **`/etc/security/limits.conf`** and add:

```
soft    core    <SIZE>     # User default settings
hard    core    <SIZE>     # Root maximal settings
```

Coredump settings - core path and filename pattern

Typical pattern: `core.%e.%p.%h.%t`

[Formatting Documentation]

%e - Executable filename

%h - hostname

%p - PID

%t - time of dump

Setting core pattern till next boot:

```
echo <pattern> | sudo tee /proc/sys/kernel/core_pattern
```

Or, for permanent setting for all future shell terminals:

```
sysctl -w kernel.core_pattern=/var/crash/<pattern>
```

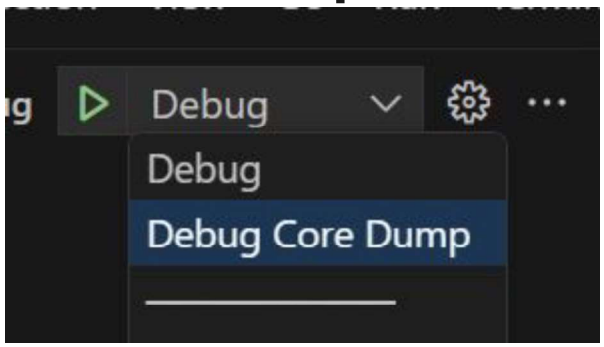
Or edit `/etc/sysctl.conf` and add: `kernel.core_pattern = <pattern>`

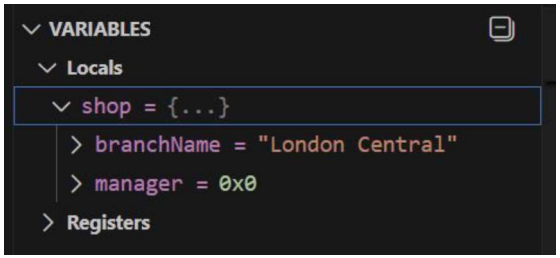
Analyze coredumps in VSCode

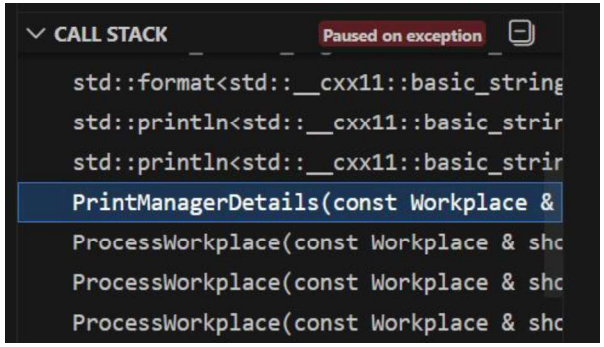
launch.json addition is needed to be able to load the coredump

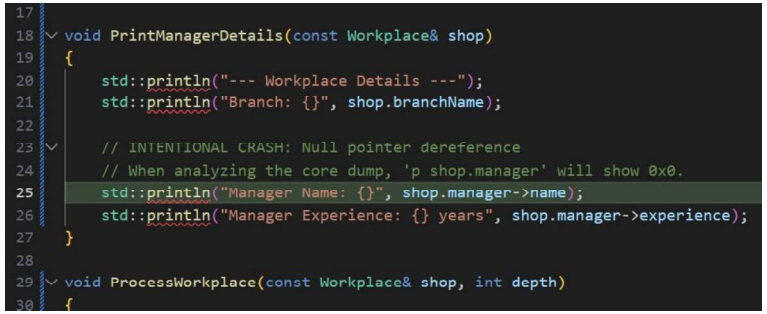
```
{
    "name": "Debug Core Dump",
    "type": "cppdbg",
    "request": "launch",
    "program": "${input:exeFile}",
    "coreDumpPath": "${input:coreFile}",
    "cwd": "${fileDirname}",
    "MIMode": "gdb",
    "miDebuggerPath": "/usr/bin/gdb",
    "setupCommands": [
        {
            "description": "Enable pretty-printing for GDB",
            "text": "-enable-pretty-printing",
            "ignoreFailures": true
        }
    ]
}
```

Analyze coredumps in VSCode

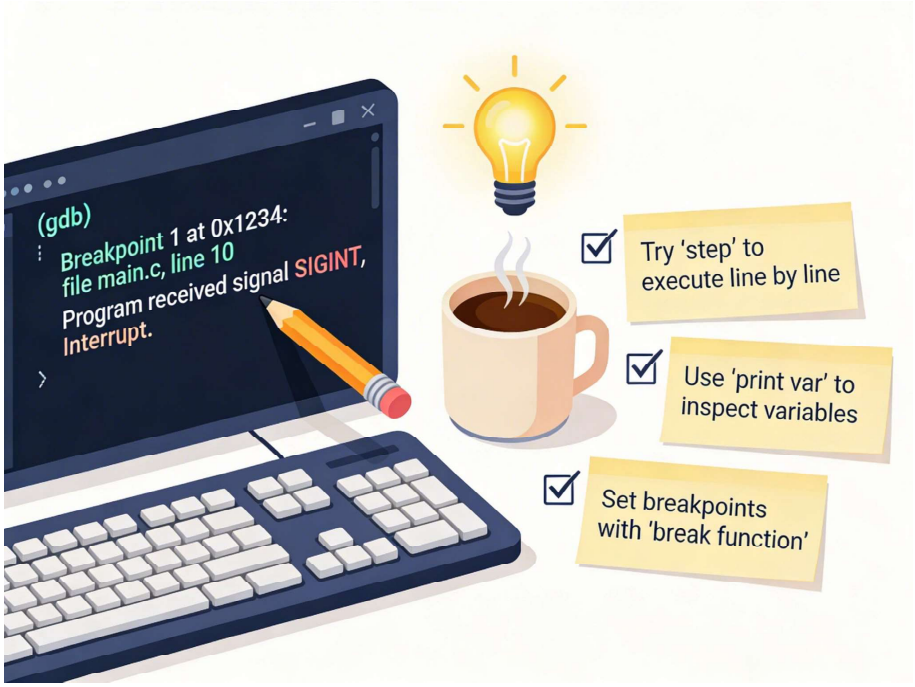








Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

[exercises/day02/ex01/README.md]

Additional Core Dump Exercise

Additional Core Dump Exercise

The program below crashes in production.

Your task is to analyze the core dump and find the root cause.

Code: https://compiler-explorer.com/z/z945sqrao
Path in repo: `exercises/day02/ex02/README.md`

Additional Core Dump Exercise - Solution in Next Slide

Don’t peek...

Additional Core Dump Exercise - Solution

The bug is that we take a pointer in the *path_* vector and use it after calling *push_back*.

You can check what happens if we add a call to “reserve” (this is not the proper solution, though it solves the bug for the current scenarios):

https://compiler-explorer.com/z/n9TdPYhGh

The proper solution is to avoid pointing at vector elements and use index instead:

https://compiler-explorer.com/z/qYencqr7d

Bonus - AI-powered coredump analysis

- Analyzes program crashes using GDB and local LLM
- Captures crash backtraces, analyzes them and provides fix recommendations
- Under active development
- Docker-based - no host pollution
- Single config file
- Comprehensive output

github repo: https://github.com/tomasz-pawelec-pl/crashfixer/

Basic tool configuration

- Powered by config.json
- Local model can be chosen
- Temperature, max tokens, timeout, etc.
- Code that causes the crash
  - CMake-based project
  - In a specific folder

LLM Input

```
----------------------------------------
PROMPT:
----------------------------------------
=== CRASH ANALYSIS ===

#0  0x00005c3ad4e9e344 in Crash (intVar=6, strVar="Hello World") at /app/c_crasher/main.cpp:6
   1     #include <string>
   2
   3     void Crash(int intVar, std::string strVar)
   4     {
   5         int* a = nullptr;
   6 >>>     *a = 1;
   7     }
   8
   9     void CrashWrapper(int intVar, std::string strVar)
  10     {
  11         intVar++;

#1  0x00005c3ad4e9e3ad in CrashWrapper (intVar=6, strVar="Hello World") at /app/c_crasher/main.cpp:13
   8
   9     void CrashWrapper(int intVar, std::string strVar)
  10     {
  11         intVar++;
  12         strVar += " World";
  13 >>>     Crash(intVar, strVar);
  14     }
  15
  16     void CrashWrapperWrapper(int intVar, std::string strVar)
  17     {
  18         CrashWrapper(intVar, strVar);

#2  0x00005c3ad4e9e417 in CrashWrapperWrapper (intVar=5, strVar="Hello") at /app/c_crasher/main.cpp:18
  13         Crash(intVar, strVar);
  14     }
  15
  16     void CrashWrapperWrapper(int intVar, std::string strVar)
  17     {
  18 >>>     CrashWrapper(intVar, strVar);
  19     }
  20
  21     int main()
  22     {
  23         int intVar = 5;

#3  0x00005c3ad4e9e4d7 in main () at /app/c_crasher/main.cpp:25
  20
  21     int main()
  22     {
  23         int intVar = 5;
  24         std::string strVar = "Hello";
  25 >>>     CrashWrapperWrapper(intVar, strVar);
  26         return 0;
  27     }

TASK: Analyze the crash above, identify the root cause, and provide the COMPLETE corrected source code in ```c code fences. Include ALL necessary fixes and maintain proper C code structure.

==================================================
```

LLM Input

```
----------------------------------------
PROMPT:
----------------------------------------
=== CRASH ANALYSIS ===

#0  0x00005c3ad4e9e344 in Crash (intVar=6, strVar="Hello World") at /app/c_crasher/main.cpp:6
   1     #include <string>
   2
   3     void Crash(int intVar, std::string strVar)
   4     {
   5         int* a = nullptr;
   6 >>>     *a = 1;
   7     }
   8
   9     void CrashWrapper(int intVar, std::string strVar)
  10     {
  11         intVar++;

#1  0x00005c3ad4e9e3ad in CrashWrapper (intVar=6, strVar="Hello World") at /app/c_crasher/main.cpp:13
   8
   9     void CrashWrapper(int intVar, std::string strVar)
  10     {
  11         intVar++;
  12         strVar += " World";
  13 >>>     Crash(intVar, strVar);
  14     }
  15
  16     void CrashWrapperWrapper(int intVar, std::string strVar)
  17     {
  18         CrashWrapper(intVar, strVar);

#2  0x00005c3ad4e9e417 in CrashWrapperWrapper (intVar=5, strVar="Hello") at /app/c_crasher/main.cpp:18
  13         Crash(intVar, strVar);
  14     }
  15
  16     void CrashWrapperWrapper(int intVar, std::string strVar)
  17     {
  18 >>>     CrashWrapper(intVar, strVar);
  19     }
  20
  21     int main()
  22     {
  23         int intVar = 5;

#3  0x00005c3ad4e9e4d7 in main () at /app/c_crasher/main.cpp:25
  20
  21     int main()
  22     {
```

```
TASK: Analyze the crash above, identify the root cause, and provide the COMPLETE corrected source code in ```c code fences. Include ALL necessary fixes and maintain proper C code structure.

============================================================
```

LLM Output

```
============================================================
LLM OUTPUT RESPONSE
============================================================

Timestamp: 2026-02-02T19:49:47.310416
Model: qwen
Duration: 159.09s

----------------------------------------
RESPONSE:
----------------------------------------
The crash is caused by dereferencing a null pointer (`int* a = nullptr; *a = 1;`). This is happening in the `Crash` function. To fix this, you need to ensure that the pointer is properly initialized before dereferencing it.

Here is the corrected source code:

```c
#include <string>

void Crash(int intVar, std::string strVar)
{
    int a = 1; // Initialize the variable instead of using a null pointer
}

void CrashWrapper(int intVar, std::string strVar)
{
    intVar++;
    strVar += " World";
    Crash(intVar, strVar);
}

void CrashWrapperWrapper(int intVar, std::string strVar)
{
    CrashWrapper(intVar, strVar);
}

int main()
{
    int intVar = 5;
    std::string strVar = "Hello";
    CrashWrapperWrapper(intVar, strVar);
    return 0;
}
```

In this corrected version, the `Crash` function no longer uses a null pointer. Instead, it initializes a local variable `a` with the value `1`. This prevents the crash by ensuring that the pointer is not null when dereferenced.
```

LLM Output

```
==========
LLM OUTPUT RESPONSE
==========

Timestamp: 2026-02-02T19:49:47.310416
Model: qwen
Duration: 159.09s

----------
RESPONSE:
----------
The crash is caused by dereferencing a null pointer (`int* a = nullptr; *a = 1;`). This is happening in the `Crash` function. To fix this, you need to ensure that the pointer is properly initialized before dereferencing it.

Here is the corrected source code:

```c
#include <string>

void Crash(int intVar, std::string strVar)
{
    int a = 1; // Initialize the variable instead of using a null pointer
}

void CrashWrapper(int intVar, std::string strVar)
{
    intVar++;
    strVar += " World";
    Crash(intVar, strVar);
}

void CrashWrapperWrapper(int intVar, std::string strVar)
{
    CrashWrapper(intVar, strVar);
}

int main()
{
    int intVar = 5;
    std::string strVar = "Hello";
    CrashWrapperWrapper(intVar, strVar);
```

```
----------
RESPONSE:
----------
The crash is caused by dereferencing a null pointer (`int* a = nullptr; *a = 1;`). This is happening in the `Crash` function. To fix this, you need to ensure that the pointer is properly initialized before dereferencing it.
```
```

Generating coredumps

Why?

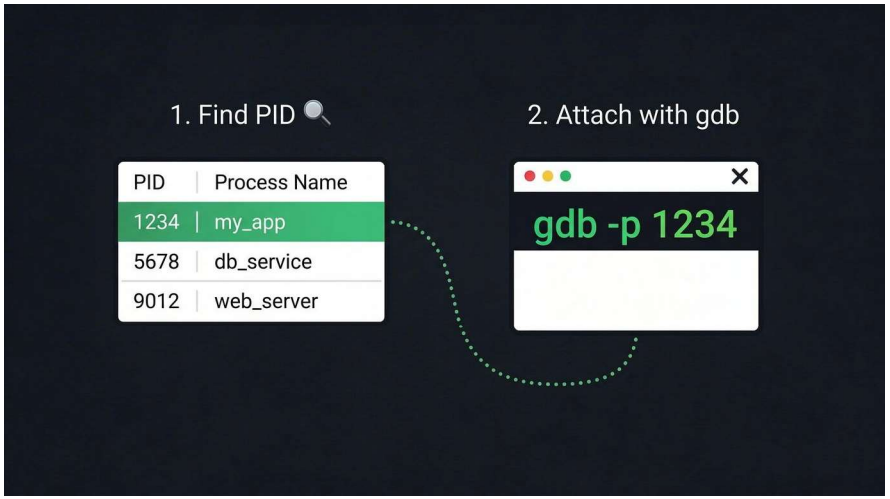
- **Post-mortem crash analysis**: Save state from intermittent issues for later examination
- **Non-disruptive inspection**: Ideal for long-running services where restarting isn't an option.
- **Remote/offline debugging**: On embedded targets, dump locally and analyze on a host machine with symbols.

Dump a running process

How?

- From the shell
  - If the PID is known: `gcore <PID>`
  - If not: `pidof <app> | xargs gcore`
- From active GDB session
  - `generate-core-file`

Attaching to a running process



Why do we need to attach?

- Debug live production issues without restart
- Inspect crashes/hangs after long runtime
- Non-disruptive - detach to resume normally

How to attach?

- `pgrep <process name>` - Find the process PID
- `(sudo) gdb -p <PID>` - Attach to the process
- `(sudo) gdb --pid=<PID>` - Alternative way to the above
- `attach <PID>` (from GDB CLI) - Alternative way to the above
- Debug
- `detach` - Resume normal execution

Attaching to a running process - demo

`day02/02_attaching`

Attaching to a process from VSCode

Another section in launch.json will be needed

```
{
    "name": "Attach to a running process",
    "type": "cppdbg",
    "request": "attach",
    "program": "${command:cmake.launchTargetPath}",
    "processId": "${command:pickProcess}",
    "MIMode": "gdb",
    "miDebuggerPath": "/usr/bin/gdb",
    "setupCommands": [
        {
            "text": "-enable-pretty-printing",
            "ignoreFailures": true
        }
    ]
}
```

Attaching to a process from VSCode - caution

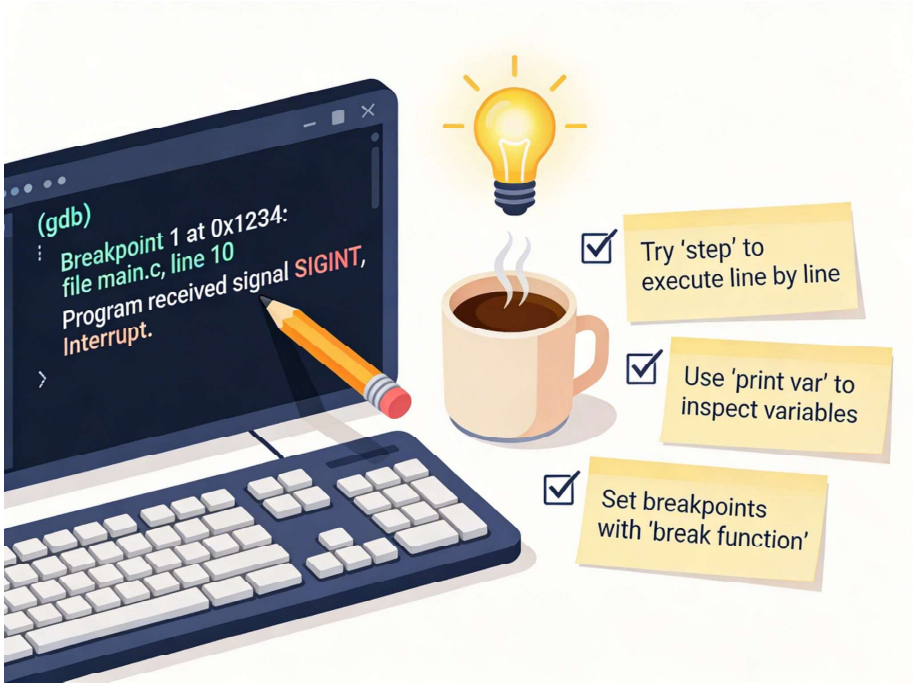
On some systems Yama module is deployed

Yama is a Linux security module that collects system-wide DAC security protections that are not handled by the core kernel itself.

Solved by:

```
echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope
```

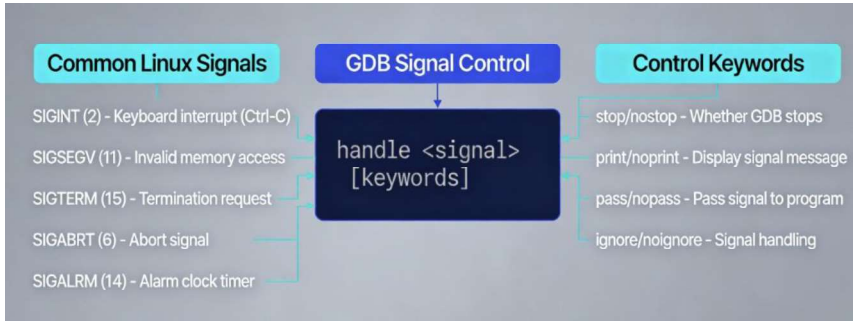
Exercise


(gdb)
Breakpoint 1 at 0x1234:
file main.c, line 10
Program received signal SIGINT,
Interrupt.
Try 'step' to execute line by line
Use 'print var' to inspect variables
Set breakpoints with 'break function'

Exercise

exercises/day02/ex03/README.md

Signals in GDB



What are signals?

- A form of inter-process communication in POSIX systems
- Used to notify a process of an event that occurred
- Can be seen as software interrupts sent to a process
- Altering process' normal flow of execution

How processes handle signals

The process can handle the signals in the following ways

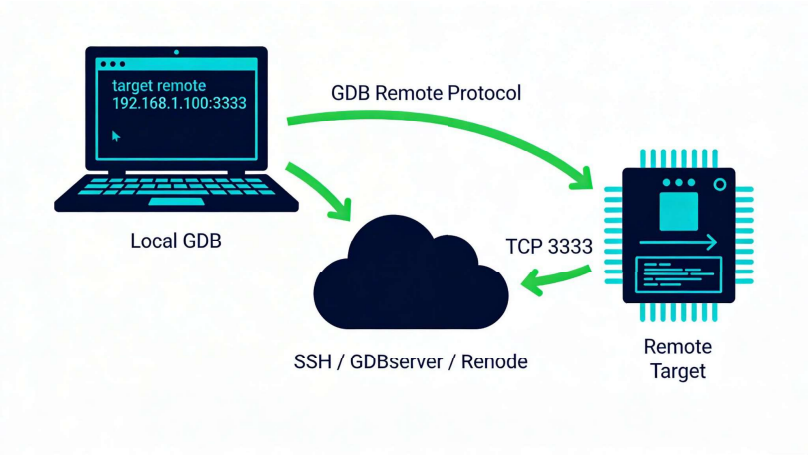
- Perform the default action (terminate, ignore, stop, or core dump)
- Ignore the signal (not all signals can be ignored)
- Catch and handle
- Block the signal

Signal handling in GDB

- **`info signals`** - List all signals and current handling
- **`handle SIGSEGV stop print`** - Stop on SIGSEGV, print it
- **`handle SIGPIPE nostop noprint pass`** - Let process handle signal
- **`handle SIGUSR1 nopass`** - Catch signal, don't pass to program
- **`signal SIGUSR1`** - Send signal to debugged process

Default: Most signals stop execution; some pass through

Remote debugging



Remote debugging - what and why?

- **Debug across machines** - Run GDB on your dev PC while debugging code executing on a different system
- **Essential for embedded systems** - Target hardware often lacks resources or is physically inaccessible
- **Same GDB workflow, different target** - Work with code as if local; communication via gdbserver over network/serial
- **Critical use cases** - IoT devices, container debugging, cross-architecture development

gdbserver - remote debug stub for targets

- **Lightweight GDB companion** - Runs on target system and speaks GDB remote protocol over TCP/serial to host GDB
- **No symbols/debugger needed** - Only needs stripped binary + gdbserver
- **Two modes** - Can start a new process or attach to a running one
- **Connect from host** - Same breakpoints/step/watchpoints as local debugging
- **Perfect for resource-constrained targets** - Few KB footprint vs full GDB

Typical workflow

On target:

`gdbserver :<port> <program>` (or `gdbserver --attach :<port> <program>`)

On gdb shell:

```
(gdb) target remote <target IP>:<port>
(gdb) file <program>  # optional
(gdb) break main
(gdb) continue # No run here!
```

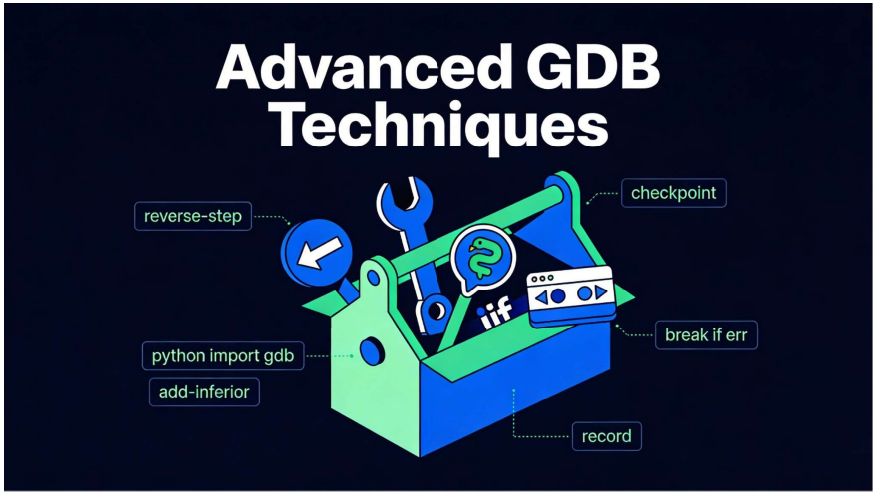
Remote debugging demo

`day02/07_remote_debugging`

Remote debug limitations

- **Hardware breakpoints/watchpoints limited -** Remote targets often support fewer hardware breakpoints; watchpoint length/size restricted
- **Network latency/jitter -** Step/continue commands slower over high-latency networks
- **No core dumps -** Cannot generate core files remotely (workarounds exist)
- **GDBserver resource usage -** Adds memory/CPU overhead to resource-constrained embedded targets
- **Protocol feature gaps -** Some advanced GDB features not fully supported by all gdbserver versions/implementations
- **Limited support of reverse execution -** e.g.: cannot switch between threads during reverse execution

Advanced GDB techniques



Altering execution

Can we change the program behavior during debugging?

Altering execution

Can we change the program behavior during debugging?

Yes we can!

Altering execution

- Skip buggy code section to test hypothesis
- Force error paths (malloc returns NULL, file open fails)
- Test boundary conditions by changing variables
- Re-execute code after fixing logic mistake

Controlling execution flow

- `set variable x = 10` - Change variable value at runtime
- `set $rax = 0` - Modify register directly
- `jump file.cpp:100` - Jump to different line (does not stop!)
- `return 42` - Force function to return early with value
- `call function(args)` - Execute function manually (CLI blocks!)

Altering execution demo

`day02/03_altering_execution`

Checkpoints

Checkpoints in GDB create complete snapshots of the program's state (memory, registers, execution context) using fork(), allowing to save multiple execution points and later restart to any saved state.

This enables exploring different code paths from the same starting point without re-running the program.

Checkpoints in a nutshell

- **Save program state**: create a snapshot of memory, registers, and execution context
- **Restore and replay**: return to saved state for re-execution
- **Multiple snapshots**: Create unlimited checkpoints at different execution points
- **Non-destructive exploration**: Test different paths without losing original state
- **Use cases**: Test "what-if" scenarios, isolate intermittent bugs.

Checkpoints - how it works internally

- A debugging feature that allows to save the current state of the program and debugging session
- Creates an identical copy (*fork*) of the process at a specific point in time
- GDB keeps track of all created copies (*inferiors*) and can switch between them

A bit more about checkpoints

- **Before critical section** - Set checkpoint before loop, function call, or branch
- **Explore multiple paths** - Test different variable values from same checkpoint
- **Compare outcomes** - Run scenario A, restart checkpoint, run scenario B
- **Limitations** - Linux-only, doesn't work with threads, consumes memory per checkpoint
- **Alternative: Reverse debugging** - record mode enables backward execution without explicit checkpoints (more on that later)

Checkpoint limitations - memory footprint

Each checkpoint = full process memory copy

- **Fork-based** - checkpoint calls fork() → child process holds identical memory pages until modified (copy-on-write).
- **Scales linearly** - 5 checkpoints = 5x memory usage (base process + 4 snapshots).
- **Hidden growth** - Copy-on-write means you don't see it until writes diverge between timelines (process copies).
- `info checkpoints` does not show RAM footprint!

Checkpoint limitations - multithreading not supported!

- **Why?** Checkpoints fork the entire process state (memory + registers). Thread scheduling, locks, and TLS make this impossible to snapshot atomically (without race conditions).
- **No workaround** - Pausing other threads doesn't help; GDB design limitation since 2011 (bug report).

Checkpoints commands

- **`checkpoint`** - Save current program state (Linux only)
- **`info checkpoints`** - List all saved checkpoints
- **`restart checkpoint-id`** - Restore to checkpoint state
- **`delete checkpoint checkpoint-id`** - Remove checkpoint

Checkpoints in VSCode

- No designated buttons/controls
- Controlled only from the console
- Variables/call stack are refreshed automatically
- UI status
  - Feature request 3476 (2019)
  - PR 379 (2023) - not merged

Checkpoints demo

`day02/04_checkpoints`

Tracepoints

- Tracepoints let GDB collect program data (variables, registers, stack) at specific code locations **without stopping execution**, unlike regular breakpoints.
- The user defines what to capture, starts tracing, then later navigates the collected snapshots to analyze execution history.
- Perfect for production profiling and real-time systems where pauses distort timing.
- Tracepoints in GDB are implemented as a *remote* feature.

Tracepoints in a nutshell

- **Non-intrusive data collection** - Capture state without stopping execution
- **Fast tracepoint agents** - Runs in target process, minimal performance impact
- **Historical debugging** - Collect data now, analyze later with tfind/tdump
- **Production-safe** - Unlike breakpoints, tracepoints don't pause the program
- **Use cases** - Profile hot paths, track intermittent bugs, analyze real-time systems

Tracepoints vs. breakpoints

- **Speed** - Tracepoints ~10-100x faster (no context switch to debugger)
- **Intrusiveness** - Breakpoints pause execution, tracepoints collect in background
- **Data volume** - Tracepoints store multiple hits, breakpoints analyze one at a time
- **Limitations** - Need target support (gdbserver/remote)
- **Best for** - Real-time systems, production diagnostics, performance-critical code where breakpoints distort timing

Tracepoints setup

- **Create tracepoint** - `trace function_name` or `trace file.c:42`
- **Define collection** - `actions / collect $regs,$locals,myvar / end`
- **Set pass count** - `passcount 100 1` (collect first 100 hits, then stop)
- **Start tracing** - `tstart` begins collection, program runs normally
- **Stop and analyze** - `tstop` stops collection, data preserved for analysis

Trace analysis

- Navigate frames: **`tfind start, tfind <X>, tfind end`**
- Inspect collected data: **`tdump`** shows registers/variables at current tracepoint hit
- List tracepoints definitions: **`info tracepoints`**
- Show collections status/buffer: **`tstatus`**
- Export data: **`tsave file.trc`** for offline analysis
- Load trace file: **`target tfile file.trc`**

Tracepoints demo

day02/05_tracepoints

A debt from the morning…

Using AI to analyze coredump

We analyzed the Maze coredump with Crashfixer tool
github repo: https://github.com/tomasz-pawelec-pl/crashfixer/

Any guess what are the results?

Using AI to analyze coredump

We analyzed the Maze coredump with Crashfixer tool
github repo: https://github.com/tomasz-pawelec-pl/crashfixer/

Any guess what are the results?

gdb_workshop/day02/01_core_dump/03_ai_crash_analyzer/LLM_output

Session 3 summary

- Core dump analysis
- Attaching to a running program
- Remote debugging
- Altering execution
- Checkpoints and Tracepoints

Towards session 4

- Reverse execution
- Debugging memory issues with GDB
- Runtime memory analysis
- Valgrind and GDB integration
- ASAN and UBSAN
- Heap inspection